МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

ПРОТОКОЛ

лабораторної роботи №3

«Надійність виклику інструментів і трасування агента: перевірки викликів, Pydantic, Langfuse»

Виконав(-ла):
Студент групи AI-242
Левентій Дмитро Олександрович

Перевірив: ас. Баранов О.М.

Одеса – 2026

2. Мета роботи, номер варіанта, предметна область

Мета роботи: зробити виклик інструментів надійним: перевіряти кожен виклик моделі до виконання на трьох рівнях (синтаксис, схема, зміст), повертати моделі зрозумілі повідомлення про помилки з трьох частин та бачити роботу агента як трасу в Langfuse.

Предметна область: консультант ветеринарної клініки з догляду за домашнім котом. Агент має доступ до трьох інструментів: пошук статей корпусу (search_articles), довідник щеплень (vaccine_info) та калькулятор корму (calc_food). Четвертий інструмент book_vet (запис до ветеринара) існує лише як заглушка в tools.py і агентові заборонений.

Варіант: тематика «Догляд за домашнім котом» (номер варіанта – 12). Що звіряємо з довідником: щеплення з data/vaccines.csv. Заборонена дія: book_vet.

Модель: qwen3:4b (локально, через Ollama, temperature=0.2, ліміт кроків агента – 6). Нові параметри: MAX_RETRIES = 2, ALLOWED = ["search_articles", "vaccine_info", "calc_food"]. Дані, набір задач і параметри виклику моделі порівняно з лабораторною роботою №2 не змінювалися.

3. Класи аргументів інструментів (файл arguments.py)

Правила аргументів записано один раз у класах Pydantic. З них генерується розділ parameters для моделі (schemas.py, метод model_json_schema()) і ними ж диспетчер перевіряє відповідь моделі. Скрізь стоїть extra="forbid", тому вигадані поля не відкидаються мовчки, а дають помилку.

```
"""Класи аргументів інструментів."""
from pydantic import BaseModel, ConfigDict, Field



class SearchArgs(BaseModel):
    model config = ConfigDict(extra="forbid")
    query: str = Field(min_length=2, description="Тема або проблема, одне-два слова українською,
"
                                                 "не все питання користувача цілком")



class VaccineArgs(BaseModel):
    model config = ConfigDict(extra="forbid")
    vaccine: str = Field(description="Назва щеплення в нижньому регістрі, як вона записана в
довіднику, "
                                     "наприклад: комплексне, від сказу")



class FoodArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    weight_kg: float = Field(gt=0, le=15, description="Вага кота в кілограмах")
```

```
    grams_per_kg: float = Field(gt=0, le=100, description="Норма корму в грамах на кілограм ваги
на день; "
                                                          "якщо користувач не назвав,
використовуй 25")
    days: float = Field(gt=0, le=365, description="На скільки днів рахуємо")



ARGS = {
    "search articles": SearchArgs,
    "vaccine info": VaccineArgs,
    "calc food": FoodArgs,
}
```

Перевірка згенерованої схеми (фрагмент для calc_food): у виводі є "additionalProperties": false, межі exclusiveMinimum / maximum та список required:

```
{ "additionalProperties": false,
  "properties": {
    "weight kg":    {"exclusiveMinimum": 0, "maximum": 15,  "type": "number", ...},
    "grams per kg": {"exclusiveMinimum": 0, "maximum": 100, "type": "number", ...},
    "days":         {"exclusiveMinimum": 0, "maximum": 365, "type": "number", ...}
  },
  "required": ["weight_kg", "grams_per_kg", "days"], "type": "object" }
```

4. Правила змісту

Заборону дії та перевірку ключа довідника не можна виразити схемою, тому вони винесені в код: заборона перевіряється диспетчером (рівень permission), ключ довідника – функцією check_meaning у checks.py. Усі правила зведено в табл. 1.

Таблиця 1 – Правила змісту

Аргумент	Правило	Де записано	Звідки взято межу
search_articles.query	Рядок, довжина не менше 2 символів	Клас SearchArgs (min_length=2)	Обрано самостійно: пошук за підрядком з одним символом знайшов би майже кожну статтю корпусу
vaccine_info.vaccine	Значення має бути серед назв щеплень довідника (регістр не враховується)	check_meaning (checks.py)	Ключ довідника – колонка vaccine файлу data/vaccines.csv (10 значень; вони перелічені в повідомленні для моделі)
calc_food.weight_kg	0 < вага ≤ 15	Клас FoodArgs	Обрано самостійно: вага домашнього кота, навіть найбільші породи значно легші за 15 кг
calc_food.grams_per_kg	0 < норма ≤ 100	Клас FoodArgs	Типова норма 25 г/кг (значення за замовчуванням з опису поля); 100 – чотирикратний запас, щоб не відхиляти кошенят

Аргумент	Правило	Де записано	Звідки взято межу
calc_food.days	0 < днів ≤ 365	Клас FoodArgs	Обрано самостійно: розрахунок корму не далі ніж на рік
Усі поля всіх інструментів	Вигадані поля заборонені	Класи (extra="forbid")	Вимога методички
Назва інструмента	book_vet не входить до ALLOWED	dispatcher.py (рівень permission)	Табл. 7 методички: заборонена дія варіанта

5. Перевірка ланцюжка перевірок (без моделі)

Диспетчер перевіряє виклик у такому порядку: чи існує інструмент (name), чи дозволений (permission), чи розбираються аргументи як JSON (syntax), чи відповідають класу (schema), чи мають значення сенс для даних (meaning), чи спрацював сам інструмент (tool). Для кожного рядка табл. 2 у test_checks.py викликано call_tool зі зламаним викликом.

Таблиця 2 – Перевірка ланцюжка перевірок

Що перевіряємо	Виклик	Рівень	Повідомлення для моделі
Зламаний JSON	calc_food {"weight_kg": 5, "grams_per_kg": 25, "days": 30	syntax	Помилка: аргументи не пройшли перевірку. Поле JSON: Invalid JSON: EOF while parsing an object at line 1 column 47. Виправ аргументи і виклич інструмент ще раз.
Неправильний тип	calc_food {"weight_kg": "п'ять", "grams_per_kg": 25, "days": 30}	schema	Помилка: аргументи не пройшли перевірку. Поле weight_kg: Input should be a valid number, unable to parse string as a number. Виправ аргументи і виклич інструмент ще раз.
Відсутнє обов'язкове поле	calc_food {"weight_kg": 5, "days": 30}	schema	Помилка: аргументи не пройшли перевірку. Поле grams_per_kg: Field required. Виправ аргументи і виклич інструмент ще раз.
Вигадане поле	calc_food {"weight_kg": 5, "grams_per_kg": 25, "days": 30, "x": 1}	schema	Помилка: аргументи не пройшли перевірку. Поле x: Extra inputs are not permitted. Виправ аргументи і виклич інструмент ще раз.
Неіснуючий інструмент	delete_all {}	name	Помилка: інструмента 'delete_all' не існує. Доступні: search_articles, vaccine_info, calc_food. Виклич один з доступних або повідом користувачу, що такої дії немає.
Заборонена дія	book_vet {"date": "п'ятниця"}	permission	Помилка: інструмент 'book_vet' недоступний. Не повторюй виклик, повідом користувачу, що ця дія недоступна.
Значення немає в довіднику	vaccine_info {"vaccine": "від сказу та чуми"}	meaning	Помилка: щеплення 'від сказу та чуми' немає в довіднику. Доступні: комплексне, від сказу, лейкемія котів, хламідіоз, панлейкопенія, каліцивіроз, ринотрахеїт, дерматофітоз, бордетельоз, перитоніт. Виправ назву щеплення або повідом користувачу, що даних про це щеплення немає.
Значення поза межами	calc_food {"weight_kg": 500, "grams_per_kg": 25, "days": 30}	schema	Помилка: аргументи не пройшли перевірку. Поле weight_kg: Input should be less than or equal to 15. Виправ аргументи і виклич інструмент ще раз.

Остання межа задана в класі, тому рівень schema; якби її було записано в check_meaning, рівень був би meaning. Це те саме правило змісту, перенесене в схему.

6. Результати прогонів

Базовий прогін (RUN_TAG = "baseline", лог lab3_baseline.log) виконано на агентові з лабораторної роботи №2 з підключеним трасуванням, прогін з перевірками (RUN_TAG = "checks", лог lab3_checks.log) – на тих самих п'яти задачах і з тими самими параметрами моделі.

Таблиця 3 – Результати по задачах

Задача	До: правильно (1/0)	Після: правильно (1/0)	До: кроків	Після: кроків	Що змінилося
Z1	1	1	2	2	Той самий виклик search_articles("дряпає меблі"). Змінилося лише формулювання поради; в обох прогонах порада не з корпусу (див. п. 7.4)
Z2	1	1	2	2	Без змін: vaccine_info("комплексне") → 2 міс., 650 грн
Z3	1	1	2	2	Без змін: calc_food(5, 25, 30) → 3750 г
P1	1	1	2	2	Запит до пошуку став коротшим ("проковтнув гумку" → "гумка"), результат той самий: нічого не знайдено; відповідь порадила ветеринара в обох прогонах
P2	1	1	1	1	Без змін: відмова без жодного виклику інструмента

Разом: 5/5 до і 5/5 після. Задачу Z1 зараховано, бо інструмент викликано правильно, а порада дана; але якість поради слабка в обох прогонах.

Таблиця 4 – Поламки до та після впровадження перевірок

Поламка	До	Після
Не викликав інструмент	0	0
Викликав не той інструмент	0	0
Неправильні аргументи	0	0
Зациклився	0	0
Не використав результат	0	0
Вигадав результат	0	0
Вичерпав ліміт кроків	0	0
Відмова після ліміту помилок (нове)	–	0

Примітка: у задачі Z1 інструмент повертає лише назву статті, тому порада в обох прогонах узята моделлю не з тексту статті. Як поламку «вигадав

результат» це не зараховано (виклик був, назву статті отримано), але це слабке місце, див. п. 7.4.

Таблиця 5 – Спрацювання перевірок у прогоні (lab3_checks.log)

Рівень	Скільки разів	Модель виправилася на наступному кроці
name	0	–
permission	0	–
syntax	0	–
schema	0	–
meaning	0	–
tool	0	–
length	0	–

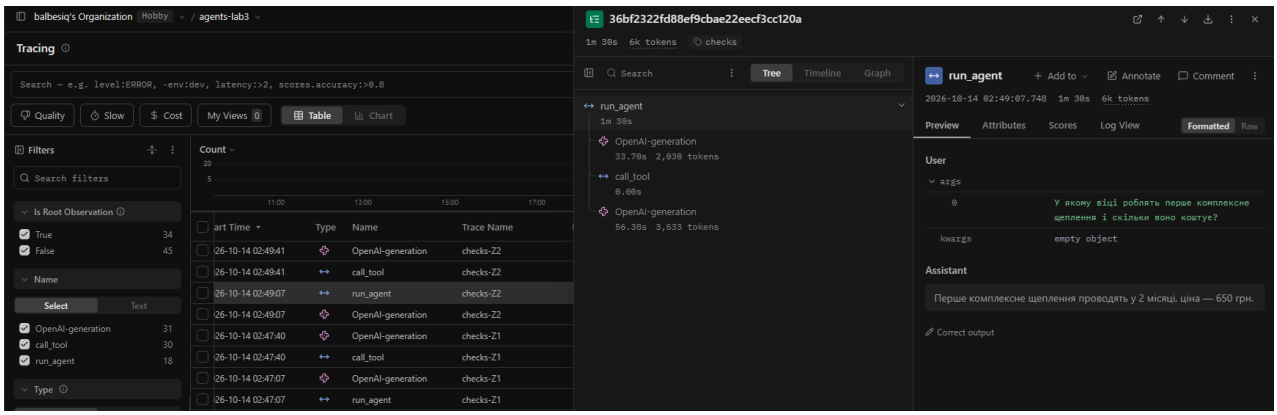
У прогоні було 4 виклики інструментів (Z1, Z2, Z3, P1), усі завершилися рівнем ok; у P2 інструмент не викликався. Помилок моделі не було, тому лічильник «помилок поспіль» у логу весь час дорівнював 0, а ліміт MAX_RETRIES жодного разу не досягався. У базовому прогоні випадків, коли відповідь моделі була схожа на виклик інструмента у вигляді тексту, теж не було.

7. Аналіз трас у Langfuse

7.1 Структура події

Розбір одного кроку агента на трасі checks-Z2 («У якому віці роблять перше комплексне щеплення і скільки воно коштує?»):

Частина	Що в трасі
Крок	Вузол OpenAI-generation: перший виклик моделі (33,7 с)
Вхід	Системне повідомлення, питання користувача та опис трьох інструментів, згенерований з класів Pydantic (additionalProperties: false, required)
Рішення	Відповідь моделі: tool_call vaccine_info з аргументами {"vaccine":"комплексне"}
Дія	Вузол call_tool: вхід ["vaccine_info", "{\"vaccine\":\"комплексне\"}"], виконався за 1 мс
Результат	Вихід call_tool: ["ok", "Перше щеплення: у 2 міс., повторювати раз на 12 міс., ціна: 650 грн"]; цей текст повертається моделі, і другий виклик моделі формує відповідь



7.2 Порівняння двох прогонів

У розділі Traces траси відфільтровано за мітками baseline і checks. У базового прогону дві серії трас (агент запускався двічі); для таблиці взято другу серію, яка відповідає lab3_baseline.log. Токени – сума total за викликами моделі, час – тривалість усієї траси.

Таблиця 6 – Порівняння прогонів у Langfuse

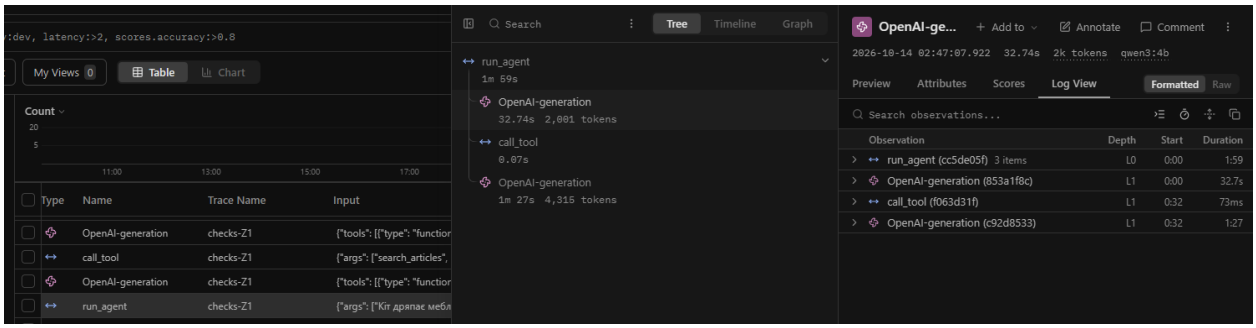
Задача	Викликів моделі: до	Викликів моделі: після	Токенів: до	Токенів: після	Час, с: до	Час, с: після
Z1	2	2	4927	6316	81.8	119.8
Z2	2	2	4156	5563	44.7	90.1
Z3	2	2	3948	3932	39.2	36.9
P1	2	2	4572	4665	61.3	68.1
P2	1	1	1613	1664	17.5	18.8
Разом	9	9	19216	22140	244.4	333.8

Кількість викликів моделі однакова (9 і 9): перевірки не додали жодного виклику, бо помилок моделі не було. Різниця в токенах і часі пояснюється недетермінованістю qwen3:4b (різна довжина міркувань і відповіді), а не перевірками: сам call_tool займає від 1 до 73 мс проти 13–120 с на траси. Для порівняння, два однакові базові прогони теж відрізняються: Z1 – 88,5 с і 81,8 с, Z2 – 70,0 с і 44,7 с, Z3 – 25,3 с і 39,2 с, сумарно 242.3 с і 244.4 с.

7.3 Траса задачі з найслабшою відповіддю (Z1)

Трасу checks-Z1 (2 виклики моделі, 6316 токенів, 119,8 с) формально зараховано, але відповідь слабка: «Рекомендується: надати підлоги для дряпі, обрізати кілки». Причина: search_articles повертає лише назви статей («Чому кіт дряпає меблі»), тож текст статті з порадою (поставити когтеточку і заохочувати кота, не карати) до моделі не доходить, і вона радить зі своїх знань, до того ж зі спотвореними словами. Жодна перевірка цього не зловила,

бо диспетчер перевіряє форму й зміст виклику (існування, дозвіл, JSON, схему, значення), а не відповідність фінального тексту джерелу. Виклик у Z1 був коректним, усі перевірки пройдено, помилка лежить у дизайні інструмента та в якості генерації моделі.



8. Відповіді на питання

1. Які поламки зникли або зменшилися після впровадження перевірок? Які лишилися?

У табл. 4 в обох прогонах усі лічильники дорівнюють 0, тому поламок, що зникли, у цьому прогоні немає: на п'яти задачах модель не припускалася помилок, які перевірки могли б зловити. Працездатність перевірок підтверджено тестом без моделі (табл. 2: усі 8 випадків спіймано на правильних рівнях). Лишилася слабка якість поради в Z1 (п. 7.4).

2. На якому рівні модель помилялася найчастіше? Що можна змінити?

У прогоні модель не помилялася (табл. 5), тому статистики за рівнями немає. У тесті табл. 2 найчастіше спрацьовував рівень schema (4 з 8 випадків). Щоб вигадані назви щеплень траплялися рідше, їх можна перенести в схему: назв рівно 10, тож підходить Literal, і модель бачитиме список допустимих значень (enum) уже в описі інструмента.

3. Чи траплялися відмови після ліміту помилок?

Ні. Лічильник помилок поспіль завжди був 0, тож ліміт MAX_RETRIES = 2 не досягався, фраза «СТОП: модель не змогла сформувати правильний виклик» у логу відсутня. Відмова в P2 не пов'язана з лімітом: модель одразу відповіла текстом, що запису немає, бо book_vet у SCHEMAS немає.

4. Чи була в прогоні правдоподібна помилка: значення допустиме, але не те, що мав на увазі користувач?

Найближчий випадок – Z1: модель викликала search_articles("дряпає меблі"), значення цілком допустиме й знайшло потрібну статтю, але фінальна порада

не відповідає тексту статті. Жодна перевірка цього не ловить, бо вона бачить лише аргументи виклику, а не те, як модель використала результат.

5. Які поламки перевірки не лікують у принципі та чому?

Перевірки працюють до виконання інструмента й дивляться на форму та значення аргументів. Тому вони не лікують вибір не того інструмента з коректними аргументами, вигадування або спотворення тексту фінальної відповіді (Z1), невикористання результату інструмента, а також повільність і недетермінованість моделі.

6. Скільки додаткових викликів моделі пішло на виправлення помилок? Чи виправдана ціна надійності?

Жодного: у табл. 6 по 9 викликів моделі в обох прогонах. Накладні витрати самих перевірок мізерні (1–73 мс на call_tool проти десятків секунд на виклик моделі). Додатковий виклик виникає лише тоді, коли модель помилилася: максимум два зайві виклики на задачу до відмови за MAX_RETRIES = 2. Така ціна виправдана.

7. Що в найдовшій задачі легше побачити в трасі Langfuse, а що в текстовому лозі?

Найдовша задача – Z1 у прогоні checks (119,8 с). У трасі видно час і токени кожного виклику моделі (близько 32,7 с і 87,0 с проти 73 мс на call_tool), повну історію повідомлень і опис інструментів на вході, а також вихід call_tool як пару (рівень, текст). У текстовому лозі це не видно: там є лише послідовність кроків, виклики й prompt_tokens. На довгих задачах лог стає важко читати, бо це лінійний текст, у якому історія повторюється з кроку в крок, а кроки не можна згорнути чи порівняти за часом.

9. Висновок

Найбільший ефект дала не окрема перевірка, а ланцюжок: Pydantic-класи як єдине джерело правил (схема для моделі й перевірка відповіді створюються з одного місця), диспетчер із рівнями name, permission, syntax, schema, meaning, tool та повідомленнями з трьох частин, а також ліміт помилок поспіль. Тест без моделі показав, що всі вісім навмисно зламаних викликів ловляться на очікуваних рівнях, а заборонений book_vet блокується диспетчером навіть без наявності в SCHEMAS.

На п'яти задачах результат до і після однаковий (5/5), бо модель не припустилася жодної помилки виклику, тому ефект перевірок у реальному прогоні не виміряно. Різницю в часі й токенах у табл. 6 викликала недетермінованість моделі, а не перевірки. Що ще варто покращити: повертати з search_articles текст або витяг зі статті, а не лише назву (причина слабкої відповіді в Z1); перенести назви щеплень у Literal; додати провокаційні питання в набір задач, щоб перевірки справді спрацьовували в прогоні; дослідити structured output як спосіб запобігати помилкам ще під час генерації.